

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: *Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*
(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour

Total Marks: 50

Annexure A

Analytical Problem Solving

Code of Conduct:

I **BALAJI D** (Name) / **192512368** (Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Department: **B.E ECE**

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **BALAJI D**

Q1. Water Jug Problem (4-gallon & 3-gallon jugs → measure exactly 2 gallons)

Problem Formulation

- State: (x, y) where x = water currently in the 4-gallon jug, y = water currently in the 3-gallon jug.
- Initial state: (0, 0) – both jugs empty.
- Goal state: (2, y) – the 4-gallon jug contains exactly 2 gallons, y can be anything.
- Operators: Fill(4-jug), Fill(3-jug), Empty(4-jug), Empty(3-jug), Pour(4→3-jug), Pour(3→4-jug).

Step-by-Step Solution

Step 1: Fill the 4-gallon jug → State: **(4, 0)**

Step 2: Pour water from the 4-gallon jug into the 3-gallon jug → State: **(1, 3)**

Step 3: Empty the 3-gallon jug → State: **(1, 0)**

Step 4: Pour the remaining 1 gallon from the 4-gallon jug into the 3-gallon jug → State: **(0, 1)**

Step 5: Fill the 4-gallon jug again → State: **(4, 1)**

Step 6: Pour from the 4-gallon jug into the 3-gallon jug until the 3-gallon jug is full (it needs only 2 more) → State: **(2, 3)**

Result: the 4-gallon jug now holds exactly 2 gallons of water.

Application – Python (BFS state-space search)

```
from collections import deque

def water_jug_bfs(cap1=4, cap2=3, target=2):
    start = (0, 0)
    visited = {start}
    parent = {start: None}
    queue = deque([start])

    while queue:
        x, y = queue.popleft()
        if x == target or y == target:
            path = []
            state = (x, y)
            while state is not None:
                path.append(state)
                state = parent[state]
            return path[::-1]

        next_states = [
            (cap1, y), (x, cap2),          # Fill jug1 / jug2
            (0, y), (x, 0),                # Empty jug1 / jug2
            (max(0, x-(cap2-y)), min(cap2, x+y)), # Pour 1->2
            (min(cap1, x+y), max(0, y-(cap1-x))), # Pour 2->1
        ]
        for s in next_states:
            if s not in visited:
                visited.add(s)
                parent[s] = (x, y)
                queue.append(s)

    return None

print(water_jug_bfs())
# Output: [(0, 0), (4, 0), (1, 3), (1, 0), (0, 1), (4, 1), (2, 3)]
```

Q2. Mars Rover – Intelligent Agent Analysis

i. Percepts of the Agent

- Camera images / video of the Martian terrain (rocks, craters, slopes).
- Spectrometer and sensor readings of soil / rock composition.
- GPS-like positional data and inertial navigation readings.
- Obstacle-detection (LIDAR / stereo-vision) data.
- Atmospheric data – temperature, dust, wind.
- Battery / solar-panel power level.
- Command / communication signals received from Earth or an orbiter.

ii. Characterization of the Operating Environment

- Partially Observable – the rover can only sense its immediate surroundings, not the whole planet.
- Stochastic – outcomes of actions (e.g., wheel slip on sand) are not fully predictable.
- Sequential – current decisions (path taken) affect future options.
- Dynamic – weather, dust storms and lighting change over time, independent of the rover.
- Continuous – position, terrain angle and sensor readings are continuous-valued.
- Unknown – the exact terrain map is not known in advance and must be discovered.

iii. Actions Available to the Agent

- Move forward / backward, turn left / right.
- Take and analyse a soil or rock sample (drill, scoop).
- Capture photographs / panoramic images.
- Transmit collected data to Earth / orbiter.
- Reposition solar panels / enter low-power (safe) mode.
- Re-plan route to avoid a newly detected obstacle.

iv. Performance Measure

- Distance safely traversed without collision or getting stuck.
- Quantity and scientific quality of samples analysed / images captured.
- Accuracy and completeness of data transmitted to mission control.
- Energy efficiency (power consumed vs. power generated).
- Overall mission time and number of objectives completed.

v. Suitable Agent Architecture

A Model-Based, Utility-Based (Goal-Based) agent is the most suitable architecture for the Mars Rover. Because the environment is only partially observable and dynamic, the agent must maintain an internal model / map of the terrain it has already explored (model-based component). Because several competing objectives exist – maximise scientific data collected, minimise energy used, avoid damage – the agent must weigh trade-offs using a utility function rather than a single fixed goal (utility-based component). A simple reflex agent is not suitable since it cannot handle the partial observability and long-term planning that Mars exploration demands.

Q3. 8-Queens Problem – Search Formulation & Solution

Problem Components

- Initial State: an empty chessboard, or one queen already placed in the first column as given (which is NOT a valid solution because it lies on the same diagonal as the queen in the last column).
- State: any arrangement of 0 to 8 queens on the board, one per column, with no two queens in the same row.
- Successor Function: add a queen to the left-most empty column, in every row that is not attacked by an existing queen.
- Goal Test: 8 queens are placed on the board such that no two queens share the same row, column, or diagonal.
- Path Cost: not relevant here (any solution is equally acceptable) – this is a constraint-satisfaction / pure search problem.

Why the Given Configuration Fails

In the configuration shown, the queen placed in the first column and the queen placed in the last column lie on the same diagonal (their row and column indices differ by the same amount), so each attacks the other. Hence the configuration violates the goal test and is not a valid solution.

Application – Python (Backtracking Search)

```
def solve_n_queens(n=8):
    solution = [-1] * n          # solution[col] = row of queen in that column

    def is_safe(col, row):
        for c in range(col):
            r = solution[c]
            if r == row or abs(r - row) == abs(c - col):
                return False      # same row or same diagonal
        return True

    def backtrack(col):
        if col == n:
            return True
        for row in range(n):
            if is_safe(col, row):
                solution[col] = row
                if backtrack(col + 1):
                    return True
                solution[col] = -1 # undo (backtrack)
        return False

    backtrack(0)
    return solution

print(solve_n_queens())
# One valid solution (row placed in each column 0..7):
# [0, 4, 7, 5, 2, 6, 1, 3]
```

The output means: place a queen in row 0 of column 0, row 4 of column 1, row 7 of column 2, and so on – in this arrangement no two queens share a row, column or diagonal, so it satisfies the goal test.

Q4. OLA Cab Booking – Problem-Solving Agent Type & Pseudocode

Type of Problem-Solving Agent

The OLA cab-booking application is best modelled as a Goal-Based (Utility-Based) Problem-Solving Agent. The customer has a clear goal – reach the destination – and there are multiple candidate actions (cab types: mini, micro, sedan, shared, prime, etc.). The agent must search among these options and select the one that best satisfies the customer's utility (cost, comfort, waiting time), which is characteristic of a utility-based agent built on top of a goal-based / search-based reasoning process.

Pseudocode for the Problem-Solving Agent

```
function OLA_CAB_AGENT(pickup, destination, preferences):
  goal <- destination reached comfortably within budget
  cab_types <- {mini, micro, sedan, shared, prime, ...}
  available_cabs <- SEARCH_NEARBY(pickup, cab_types)

  for each cab in available_cabs:
    cab.fare      <- ESTIMATE_FARE(pickup, destination, cab.type)
    cab.eta       <- ESTIMATE_ETA(pickup, cab.current_location)
    cab.comfort   <- COMFORT_RATING(cab.type)
    cab.utility    <- UTILITY_FUNCTION(cab.fare, cab.eta,
                                       cab.comfort, preferences)

  best_cab <- argmax(available_cabs, key = cab.utility)

  if best_cab satisfies preferences.constraints:
    BOOK(best_cab)
    return SUCCESS, best_cab
  else:
    return FAILURE
```

The agent perceives pickup/destination location, available nearby cabs and traffic conditions; it searches through the space of cab-type options, evaluates each using a utility function combining fare, ETA and comfort, and acts by booking the option with the highest utility that meets the customer's constraints.

Q5. Least-Cost Delivery Route – Uniform Cost Search (UCS)

Graph (edge costs read from the network diagram)

- S – A : 1 S – C : 1 S – G : 12
- A – B : 3 A – C : 1
- B – D : 3
- C – D : 1 C – G : 2
- D – G : 3

UCS Trace (Frontier ordered by cumulative path cost)

1. Expand S (cost 0) → frontier: {A:1, C:1, G:12}
2. Expand A (cost 1) → frontier: {C:1, B:4, G:12} [S-A-C tie broken, C already 1 via S-C]
3. Expand C (cost 1) → frontier: {G:3(via C), D:2, B:4, G:12}
4. Expand D (cost 2) → frontier: {G:3(via C), B:4, G:5(via D), G:12}
5. Expand G (cost 3, path S-C-G) → GOAL REACHED

Least-Cost Path

S → C → G with total cost = 1 + 2 = 3

(For comparison: S-A-C-G costs 4, S-C-D-G costs 5, S-A-B-D-G costs 10, and the direct edge S-G costs 12 – so S→C→G with cost 3 is confirmed as the cheapest route.)

Application – Python (Uniform Cost Search)

```
import heapq

graph = {
    'S': [('A', 1), ('C', 1), ('G', 12)],
    'A': [('B', 3), ('C', 1)],
    'B': [('D', 3)],
    'C': [('D', 1), ('G', 2)],
    'D': [('G', 3)],
    'G': [],
}

def uniform_cost_search(graph, start, goal):
    frontier = [(0, start, [start])] # (cost, node, path)
    visited = set()

    while frontier:
        cost, node, path = heapq.heappop(frontier)
        if node == goal:
            return cost, path
        if node in visited:
            continue
        visited.add(node)
        for neighbour, weight in graph[node]:
            if neighbour not in visited:
                heapq.heappush(frontier,
                               (cost + weight, neighbour, path + [neighbour]))
    return float('inf'), []

cost, path = uniform_cost_search(graph, 'S', 'G')
print(f'Least cost: {cost}, Path: {" -> ".join(path)}')
# Output: Least cost: 3, Path: S -> C -> G
```

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation